การใช้งาน Arduino ESP32 Interrupt

https://www.arduinoall.com/article/164/11-esp32-สอน-arduino-esp32-การใช้งาน-arduino-esp32-interrupt

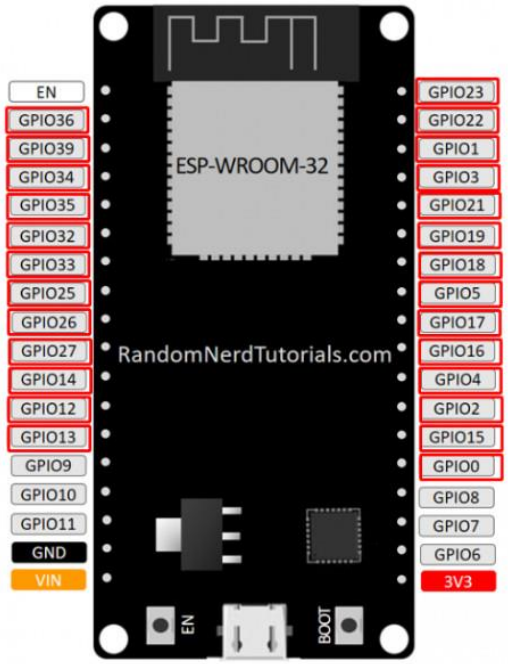
คำสั่ง Interrupt หรือการทำงานแบบขัดจังหวะ มีประโยชน์อย่างมาก ในการสั่งทำงานแบบเกิดเหตุการ เมื่อเกิดเหตุการตามที่เรากำหนดไว้ ก็จะละพักงานเดิม มาทำงานในคำสั่ง Interrupt ก่อน

เปรียบเทียบ เหมือนกับเรากำลังวาดรูปอยู่ แล้วทุก 10 วินาที ต้องวิ่งไปเช็คว่า มีไปรษณีย์มาส่งของ หรือไม่ แต่ถ้าเราแก้ไขโดย เอาออดไปวางไว้หน้าประตู แล้วก็วาดรูปโดยไม่ต้องสนใจว่ามีไปรษณีย์มาส่ง ของ ถ้ามีเสียงออด แล้วค่อยวิ่งออกไปเช็ค งานก็จะสบายมากขึ้น

โปรแกรมกำลังทำงานไฟกระพริบอยู่ แล้วต้องการอ่านค่าจากสวิตช์ปกติจะใช้คำสั่ง if เพื่อเช็คว่าสวิตช์ ถูกกดมาหรือไม่ ซึ่งโปรแกรมก็จะเสียเวลาเช็คและเขียนโคดโปรแกรมเพิ่มเพื่อให้ทำงานได้ตามต้องการ

แต่ถ้าใช้ให้ทำงานแบบ Interrupt เราสามารถเช็คจากเหตุการเมื่อสวิตช์ถูกกด ก็จะกระโดดมาทำงานใน คำสังสวิตช์ได้ทันที เมื่อทำงานเสร็จแล้ว ก็จะกลับไปทำงานต่อจากคำสั่งเดิม

ขาที่ใช้งานแบบ Interrupt ESP32 ได้มีดังนี้



ฟังก์ชั่นการตรวจจับเหตุการ Interrupt มีทั้งหมด 5 แบบดังนี้

- **LOW** เกิดเมื่อสัญญาณขา Interrupt เป็น Low
- **HIGH** เกิดเมื่อสัญญาณขา Interrupt เป็น High
- **CHANGE** เกิดเมื่อสัญญาณขา Interrupt มีการเปลี่ยนแปลงค่าแรงดัน เช่น เปลี่ยนจาก 0 เป็น 3.3V หรือจาก 3.3 เป็น 0V
- **FALLING** เกิดเมื่อสัญญาณขา Interrupt เปลี่ยนจาก High เป็น Low
- **RISING** เกิดเมื่อสัญญาณขา Interrupt เปลี่ยนจาก Low เป็น High

ESP32 Arduino: External interrupts

https://techtutorialsx.com/2017/09/30/esp32-arduino-external-interrupts/

Introduction

The objective of this post is to explain how to handle external interrupts using the ESP32 and the Arduino core.

The setup code

We will start by declaring the pin where the interrupt will be attached on a global variable. Note that depending on your ESP32 board the pin numbering of the ESP32 microcontroller and the one labeled on the board may not match. In the FireeBeetle board, the pin used below (digital pin 25) matches with the one labeled IO25/D2.

```
const byte interruptPin = 25;
```

We will also declare a counter that will be used by the interrupt routine to communicate with the main loop function and signal that an interrupt has occurred. Note that this variable needs to be declared as volatile since it will be shared by the ISR and the main code. Otherwise, it may be removed due to compiler optimizations.

```
volatile int interruptCounter = 0;
```

Additionally we will declare a counter to keep track of how many interrupts have already occurred globally since the start of the program. So this counter will be incremented each time an interrupt occurs.

```
int numberOfInterrupts = 0;
```

Finally, we will declare a variable of type portMUX_TYPE, which we will need to take care of the synchronization between the main code and the interrupt. We will see how to use it later.

```
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
```

Moving to the setup function, we start by opening a serial connection, in order to be able to output the results of our program.

```
Serial.begin(115200);
Serial.println("Monitoring interrupts: ");
```

Next, since we are going to be working with an external pin interrupt, we need to configure the previously declared pin number as an input pin. To do so we call the pinMode function, passing as argument the the number of the pin and the operating mode.

In order to know the state of the input when no electric signal is applied to our pin, we use the INPUT_PULLUP mode. So, when no signal is applied, it will be at a voltage level of VCC instead of floating, avoiding the detection of non existing external interrupts.

```
pinMode(interruptPin, INPUT_PULLUP);
```

Next we attach the interrupt to the pin with a call to the attachInterrupt function. As first argument, we pass the result of a call to the digitalPinToInterrupt function, which converts the pin number used to the corresponding internal interrupt number.

Next we pass the function that will handle the interrupts or, in other words, that will be executed when an interrupt on the specified pin occurs. We will call it handleInterrupt and specify its code later.

Finally we pass the interrupt mode, which basically specifies which type of change in the pin input signal triggers the interrupt. We will use FALLING, which means that the interrupt will occur when a change from VCC to GND is detected on the pin.

```
attachInterrupt(digitalPinToInterrupt(interruptPin), handleInterrupt, FALLING);
```

The final setup code can be seen below.

```
const byte interruptPin = 25;
volatile int interruptCounter = 0;
int numberOfInterrupts = 0;

portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void setup() {
  Serial.begin(115200);
  Serial.println("Monitoring interrupts: ");

  pinMode(interruptPin, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(interruptPin), handleInterrupt, FALLING);
}
```

The main loop

Now we will move to the main loop. There we will simply check if our interrupts counter is greater than zero. If it does, it means that we have interrupts to handle.

So, if an interrupt has occurred we first take care of decrementing this interrupts counter, signalling that the interrupt has been detected and will be handled.

Note that this counter approach is better than using a flag since if multiple interrupts occur without the main code being able to handle them all, we will not loose any events. On the other hand if we use a flag and multiple interrupts occur without the main code being able to handle them, then the flag value will keep being set to true in the ISR and the main loop handler will only interpret as if only one has occurred.

Other important aspect to keep in mind is that we should disable interrupts when writing on a variable that is shared with an interrupt. This way we ensure that there is no concurrent access to it between the main code and the ISR.

In the Arduino environment, we usually have the NoInterrupts and Interrupts function to disable and re-enable interrupts. Nonetheless, at the time of writing, these functions were not yet implemented in the ESP32 Arduino core.

So, we perform the decrement of the variable inside a critical section, which we declare using a portENTER_CRITICAL and a portEXIT_CRITICAL macro. These calls both receive as input the address of the previously declared global portMUX_TYPE variable.

```
if(interruptCounter>0){
    portENTER_CRITICAL(&mux);
    interruptCounter--;
    portEXIT_CRITICAL(&mux);
    //Handle the interrupt
}
```

After taking care of decrementing the counter, we will now increment the global counter that holds the number of interrupts detected since the beginning of the program. This variable doesn't need to be incremented inside a critical section since the interrupt service routine will not access it.

After that, we will print a message indicating an interrupt was detected and how many interrupts have happened so far. Note that sending data to the serial port should never be done inside an interrupt service routine due to the fact that ISRs should be designed to execute as fast as possible. If you do this, you will most likely run into runtime problems.

This way, in our architecture, the ISR only takes care of the simple operation of signaling the main loop that the interrupt has occurred, and then the main loop handles the rest.

You can check the full main loop code below.

```
void loop() {
  if(interruptCounter>0){
      portENTER_CRITICAL(&mux);
      interruptCounter--;
      portEXIT_CRITICAL(&mux);

      numberOfInterrupts++;
      Serial.print("An interrupt has occurred. Total: ");
      Serial.println(numberOfInterrupts);
  }
}
```

The interrupt handling function

To finish the code, we will declare our interrupt handling function. As previously mentioned, it will only take care of incrementing the global variable that is used to signalize to the main loop that an interrupt has occurred.

We will also enclose this operation in a critical section, which we declare by calling the portENTER_CRITICAL_ISR and portExit_CRITICAL_ISR macros. They also both receive as input the address of the global portMUX_TYPE variable.

This is needed because the variable we are going to use is also changed by the main loop, as seen before, and we need to prevent concurrent access problems.

Update:

The interrupt handling routine should have the IRAM_ATTR attribute, in order for the compiler to place the code in IRAM. Also, interrupt handling routines should only call functions also placed in IRAM, as can be seen here in the IDF documentation. Thanks to Manuato for point this.

The full code for the interrupt handling function is shown below.

```
void IRAM_ATTR handleInterrupt() {
  portENTER_CRITICAL_ISR(&mux);
  interruptCounter++;
  portEXIT_CRITICAL_ISR(&mux);
}
```

The final code

The final source code can be seen below. You can copy and paste it to your Arduino environment to test it.

```
const byte interruptPin = 25;
volatile int interruptCounter = 0;
int numberOfInterrupts = 0;

portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR handleInterrupt() {
  portENTER_CRITICAL_ISR(&mux);
  interruptCounter++;
  portEXIT_CRITICAL_ISR(&mux);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Monitoring interrupts: ");
  pinMode(interruptPin, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(interruptPin), handleInterrupt, FALLING);
}

void loop() {

  if(interruptCounter>0){
      portENTER_CRITICAL(&mux);
      interruptCounter--;
      portEXIT_CRITICAL(&mux);

      numberOfInterrupts++;
      Serial.print("An interrupt has occurred. Total: ");
      Serial.println(numberOfInterrupts);
  }
}
```

Testing the code

To test the code, simply upload it to your ESP32 and open the Arduino IDE serial monitor. The easiest way to trigger interrupts is to use a wire to connect and disconnect the digital pin where the interrupt was attached to GND.

Since the pin was declared as **INPUT_PULLUP**, then this will trigger a transition from VCC to GND and an external interrupt will be detected. Please be careful to avoid connecting the ground pin to the wrong GPIO and damaging the board.

You should get an output similar to figure 1, which shows the interrupts being triggered and the global counter being printed.

```
ets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0x00
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0008,len:8
load:0x3fff0010,len:160
load:0x40078000,len:10632
load:0x40080000,len:252
entry 0x40080034
Monitoring interrupts:
An interrupt has occurred. Total: 1
An interrupt has occurred. Total: 2
An interrupt has occurred. Total: 3
An interrupt has occurred. Total: 4
An interrupt has occurred. Total: 5
An interrupt has occurred. Total: 6
An interrupt has occurred. Total: 7
An interrupt has occurred. Total: 8
An interrupt has occurred. Total: 9
An interrupt has occurred. Total: 10
An interrupt has occurred. Total: 11
An interrupt has occurred. Total: 12
An interrupt has occurred. Total: 13
An interrupt has occurred. Total: 14
```

Figure 1 – Output of the interrupt handling program.

Configuring & Handling ESP32 GPIO Interrupts In Arduino IDE

https://lastminuteengineers.com/handling-esp32-gpio-interrupts-tutorial/

Often in a project you need the ESP32 to execute its normal program while continuously monitoring for some kind of event. A solution widely adopted is the use of an Interrupt.

Interrupts In ESP32

The ESP32 offers up to 32 interrupt slots for each core. Each interrupt has a certain priority level and can be categorized into two types.

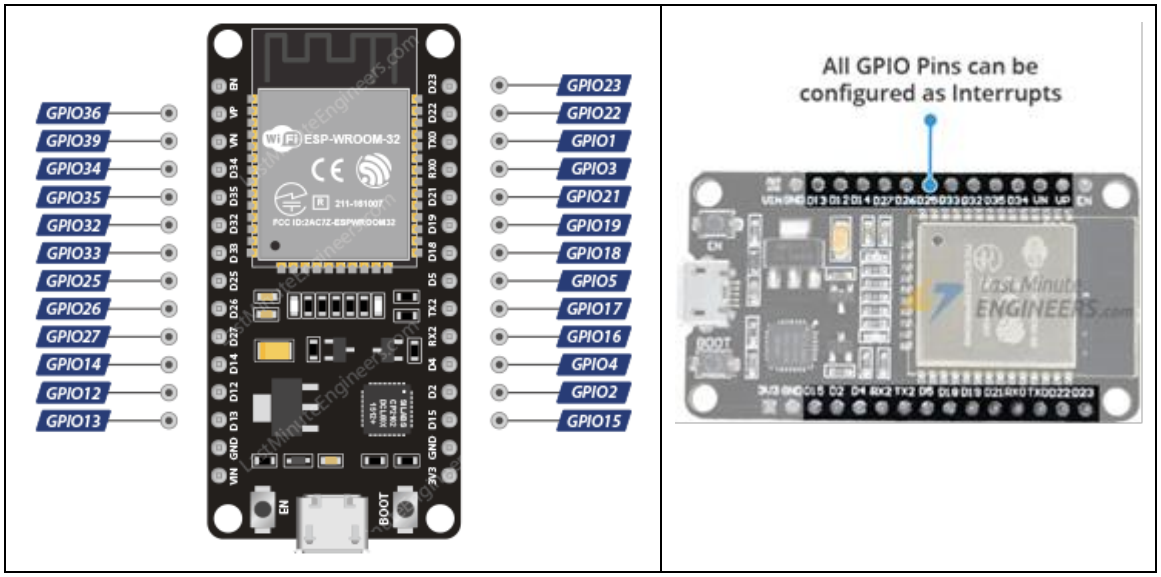
- **Hardware Interrupts** – These occur in response to an external event. For example, GPIO Interrupt (when a key is pressed down) or a Touch Interrupt (when touch is detected)

- **Software Interrupts** – These occur in response to a software instruction. For example, a Simple Timer Interrupt or Watchdog Timer Interrupt (when timer times out)

ESP32 GPIO Interrupt

In ESP32, we can define an interrupt service routine function that will be called when a GPIO pin changes its signal value.

With an ESP32 board, all the GPIO pins can be configured to function as interrupt request inputs.



Attaching Interrupt to a GPIO Pin

In Arduino IDE, we use a function called attachInterrupt() to set an interrupt on a pin by pin basis. The recommended syntax looks like below.

```
attachInterrupt(GPIOPin, ISR, Mode);
```

This function takes three parameters:

- GPIOPin – Sets the GPIO pin as an interrupt pin, which tells the ESP32 which pin to monitor.
- ISR – Is the name of the function that will be called every time the interrupt is triggered.
- Mode – Defines when the interrupt should be triggered. Five constants are predefined as valid values:

LOW	Triggers interrupt whenever the pin is LOW
HIGH	Triggers interrupt whenever the pin is HIGH
CHANGE	Triggers interrupt whenever the pin changes value, from HIGH to LOW or LOW to HIGH
FALLING	Triggers interrupt when the pin goes from HIGH to LOW
RISING	Triggers interrupt when the pin goes from LOW to HIGH

Detaching Interrupt from a GPIO Pin

You can optionally call detachInterrupt() function when you no longer want ESP32 to monitor a pin. The recommended syntax looks like below.

```
detachInterrupt(GPIOPin);
```

Interrupt Service Routine

Interrupt Service Routine is invoked when an interrupt occurs on any GPIO pin. Its syntax looks like below.

```
void IRAM_ATTR ISR() {
   Statements;
}
```

ISRs in ESP32 are special kinds of functions that have some unique rules most other functions do not have.

- The interrupt service routine must have an execution time as short as possible, because it blocks the normal program execution.
- Interrupt service routines should have the IRAM_ATTR attribute, according to the ESP32 documentation

What is IRAM_ATTR?

By flagging a piece of code with the IRAM_ATTR attribute we are declaring that the compiled code will be placed in the Internal RAM (IRAM) of the ESP32.

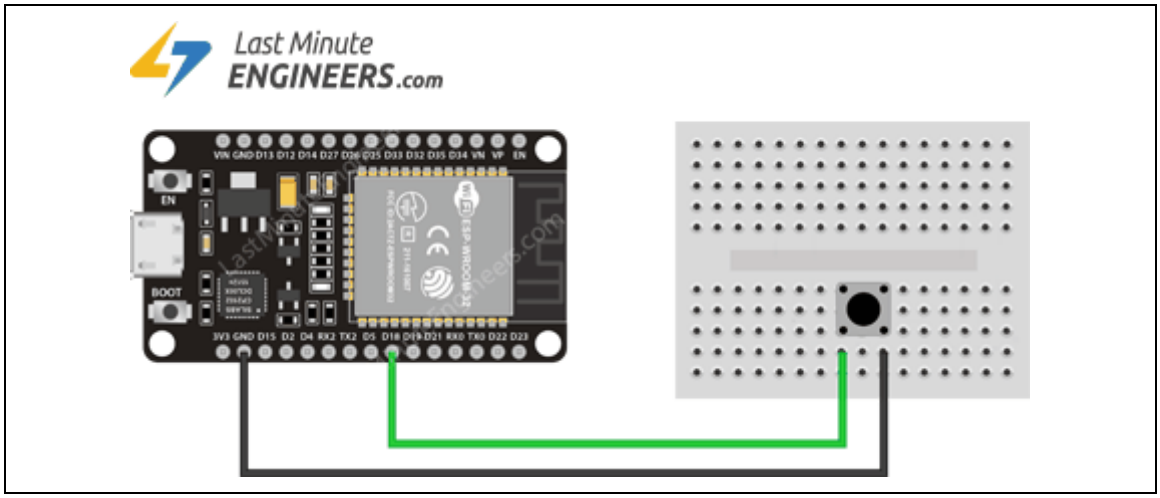
Otherwise the code is placed in the Flash. And flash on the ESP32 is much slower than internal RAM.

If the code we want to run is an interrupt service routine (ISR), we generally want to execute it as quickly as possible. If we had to 'wait' for an ISR to load from flash, things would go horribly wrong.

Hardware Hookup

Enough of theory! let's see practical example.

Let's hook one push button to GPIO pin 18 (D18) on the ESP32. You don't require any pullup to this pin as we will pull the pin up internally.

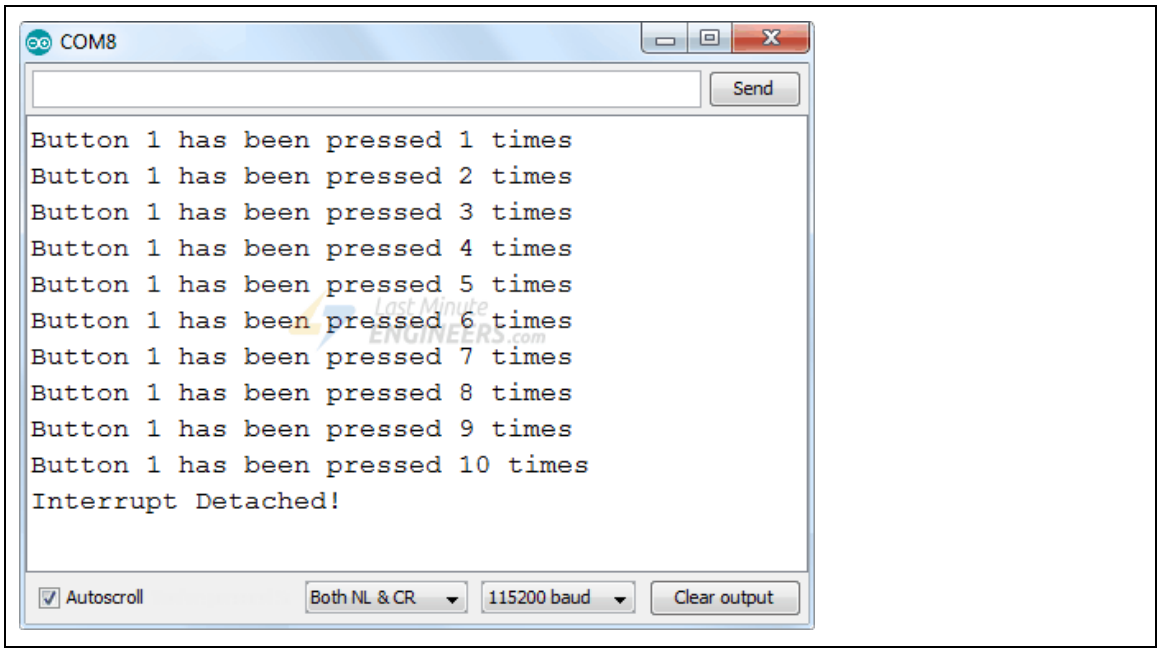


Wiring Push Buttons to ESP32 For GPIO Interrupt

Example: Simple Interrupt

The following sketch demonstrates the use of the interrupts and the correct way to write an interrupt service routine.

Once you upload the sketch, press EN button on the ESP32 and open serial monitor at baud rate 115200. Now we will get the output as shown below, when you press the button.



```
struct Button {
  const uint8_t PIN;
  uint32_t numberKeyPresses;
  bool pressed;
};

Button button1 = {18, 0, false};

void IRAM_ATTR isr() {
  button1.numberKeyPresses += 1;
  button1.pressed = true;
}

void setup() {
  Serial.begin(115200);
  pinMode(button1.PIN, INPUT_PULLUP);
  attachInterrupt(button1.PIN, isr, FALLING);
}

void loop() {
  if (button1.pressed) {
      Serial.printf("Button 1 has been pressed %u times\n",
button1.numberKeyPresses);
      button1.pressed = false;
  }



  //Detach Interrupt after 1 Minute
  static uint32_t lastMillis = 0;
  if (millis() - lastMillis > 60000) {
    lastMillis = millis();
    detachInterrupt(button1.PIN);
    Serial.println("Interrupt Detached!");
  }
}
```

Code Explanation

At the very start of the sketch we create a structure named Button. It has three members viz. pin number, number of key presses and pressed state. If you don't know, Structure is the collection of variables of different types (but logically related to each other) under a single name.

```
struct Button {
  const uint8_t PIN;
  uint32_t numberKeyPresses;
  bool pressed;
};
```

Next we create an instance of the Button structure and initialize pin number to 18, number of key presses to 0 and default pressed state to false.

```
Button button1 = {18, 0, false};
```

The following piece of code is an Interrupt Service Routine. As mentioned earlier, ISR in ESP32 should have the IRAM_ATTR attribute.
In ISR we simply increment the KeyPresses counter by 1 & set button pressed state to True.

```
void IRAM_ATTR isr() {
  button1.numberKeyPresses += 1;
  button1.pressed = true;
}
```

In Setup section of code, we first initialize the serial communication with PC. Then set input pullup the D18 pin.

Next we tell the ESP32 to monitor the D18 pin and call the interrupt service routine isr when the pin goes from HIGH to LOW i.e. FALLING state.

```
Serial.begin(115200);
pinMode(button1.PIN, INPUT_PULLUP);
attachInterrupt(button1.PIN, isr, FALLING);
```

In Loop section of the code, we simply check if the button pressed state returns to be true. When it does, we simply print the number of key pressed till now and set the button pressed state LOW so that we can continue getting next interrupts.

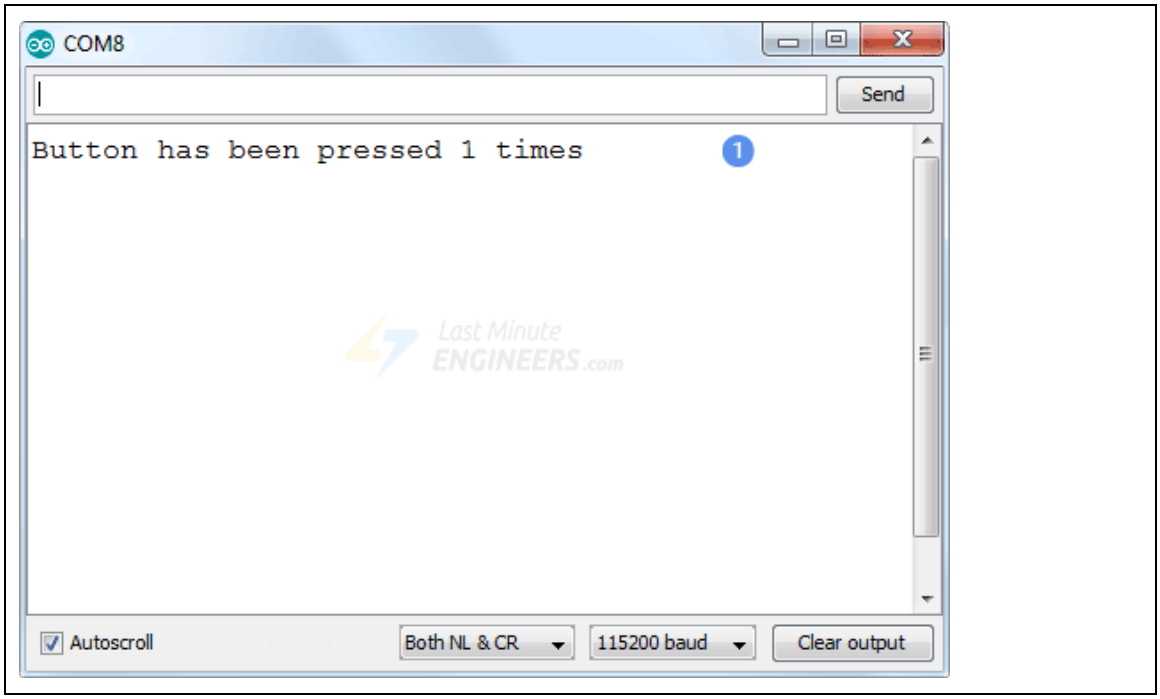
```
if (button1.pressed) {
    Serial.printf("Button 1 has been pressed %u times\n",
button1.numberKeyPresses);
    button1.pressed = false;
}
```

In loop section we also check the number of milliseconds that have passed since the program first started using millis() function. When this time is more than 60,000 milliseconds or 1 Minute, we simply tell ESP32 to not to monitor D18 pin using detachInterrupt() function.

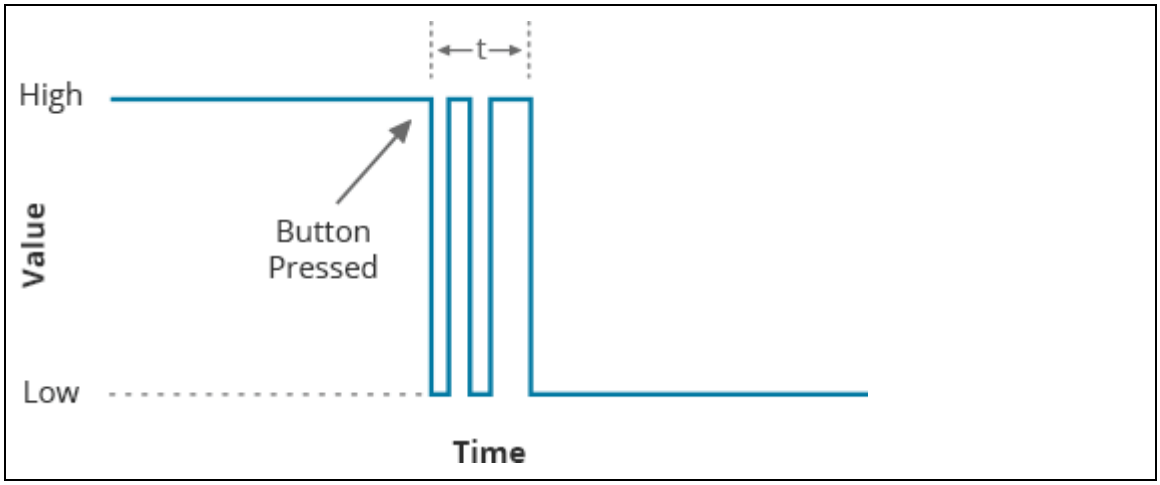
```
//Detach Interrupt after 1 Minute
static uint32_t lastMillis = 0;
if (millis() - lastMillis > 60000) {
  lastMillis = millis();
  detachInterrupt(button1.PIN);
  Serial.println("Interrupt Detached!");
}
```

Managing Switch Bounce

A common problem with interrupts is that they often get triggered multiple times for the same event. If you look at the serial output of the above example, you will notice that even if you press the button only once, the counter is incremented several times.



To find out why this happens, you have to take a look at the signal. If you monitor the voltage of the pin on the signal analyzer while you press the button, you will get a signal like this:



You may feel like contact is made immediately, but in fact the mechanical parts within the button come into contact several times before they settle into a particular state. This causes multiple interrupts to be triggered.

It is purely a mechanical phenomenon known as a ‘switch bounce’, like dropping a ball – it bounces several times before finally landing on the ground.

The time for the signal to stabilize is very fast and seems almost instantaneous to us, but for an ESP32 this is a huge period of time. It can execute multiple instructions in that time period.

The process of eliminating switch bounce is called ‘debouncing’. There are two ways to achieve this.

- Through hardware: by adding an appropriate RC filter to smooth the transition.
- Through software: by temporarily ignoring further interrupts for a short period of time after the first interrupt is triggered.

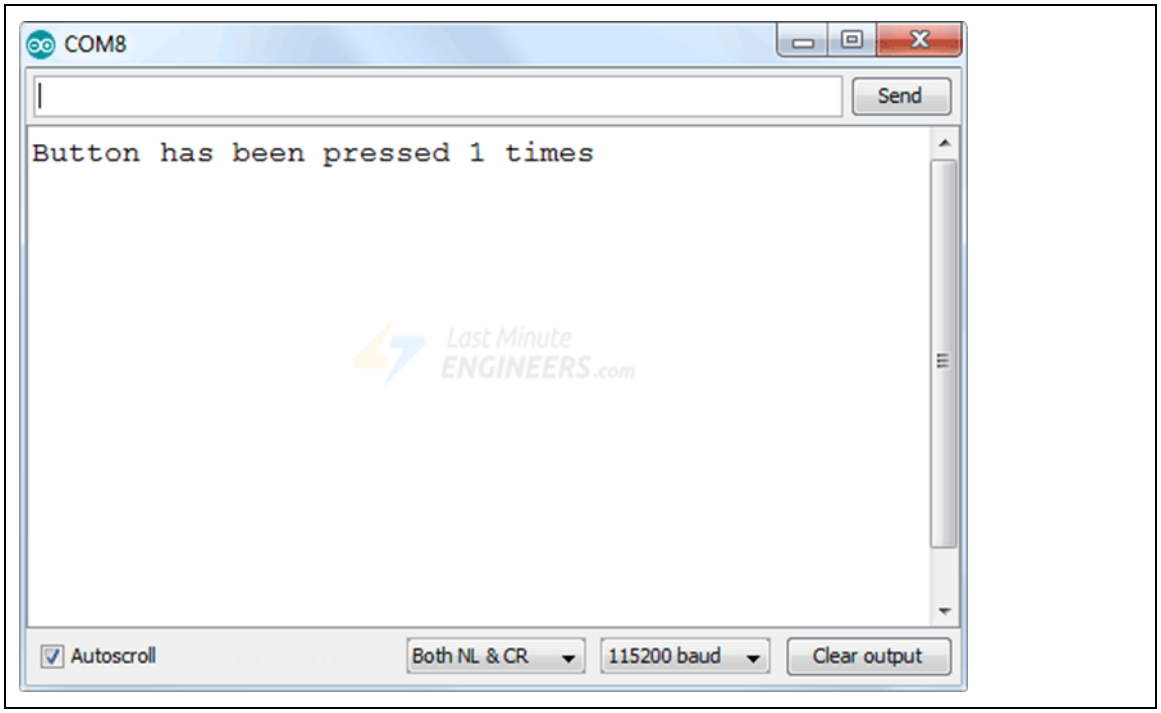
Example Code: Debouncing an Interrupt

Here the above sketch is rewritten to demonstrate how to debounce an interrupt programmatically. In this sketch we allow the ISR to be executed only once on each button press, instead of executing it multiple times.

Changes to the sketch are highlighted in green.

```
struct Button {
  const uint8_t PIN;
  uint32_t numberKeyPresses;
  bool pressed;
};

//variables to keep track of the timing of recent interrupts
unsigned long button_time = 0;
unsigned long last_button_time = 0;
Button button1 = {18, 0, false};

void IRAM_ATTR isr() {
  button_time = millis();
  if (button_time - last_button_time > 250)  {
    button1.numberKeyPresses++;
    button1.pressed = true;
    last_button_time = button_time;
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(button1.PIN, INPUT_PULLUP);
  attachInterrupt(button1.PIN, isr, FALLING);
}

void loop() {
  if (button1.pressed) {
    Serial.printf("Button has been pressed %u times\n",
button1.numberKeyPresses);
    button1.pressed = false;
  }

  //Detach Interrupt after 1 Minute
  static uint32_t lastMillis = 0;
  if (millis() - lastMillis > 60000) {
    lastMillis = millis();
    detachInterrupt(button1.PIN);
    Serial.println("Interrupt Detached!");
  }
}
```

Let's look at the serial output again as you press the button. Note that the ISR is called only once for each button press.



Code Explanation:

This fix works because each time the ISR is executed, it compares the current time returned by the `millis()` function to the time the ISR was last called.

If it is within 250ms, ESP32 ignores the interrupt and immediately goes back to what it was doing. If not, it executes the code within the `if` statement incrementing the counter and updating the `last_button_time` variable, so the function has a new value to compare against when it's triggered in the future.